

PRACTICA 007

Introducción

En esta práctica se desarrolló un programa utilizando Programación Orientada a Objetos (POO) en el lenguaje C++. El objetivo fue comprender cómo funcionan las clases, los métodos, los parámetros y los valores de retorno. El programa permite registrar y gestionar la información de un estudiante mediante un menú de opciones que permite registrar datos, modificar la calificación, consultar la información y determinar si el estudiante aprueba o reprueba.

Objetivo

Desarrollar un programa en C++ utilizando programación orientada a objetos que permita gestionar la información de un estudiante mediante métodos que reciban parámetros y devuelvan valores de retorno.

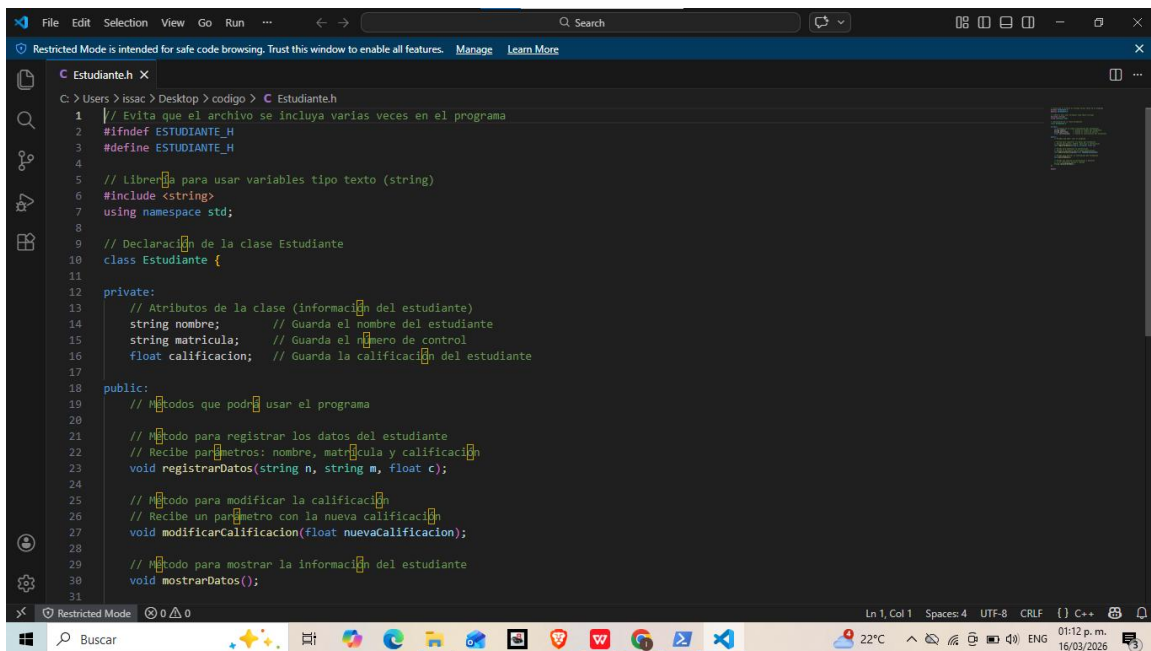
Descripción del programa

El programa representa a un estudiante mediante una clase llamada Estudiante. Esta clase contiene atributos para almacenar el nombre, la matrícula y la calificación. También contiene métodos que permiten registrar datos, modificar la calificación, mostrar la información del estudiante y determinar si el estudiante aprobó o reprobó.

Archivos del programa

1. Archivo de encabezado (Estudiante.h)

En este archivo se declara la clase Estudiante. Aquí se definen los atributos del estudiante y los métodos que pueden utilizarse. Este archivo solo describe la estructura de la clase, pero no contiene la lógica del programa.



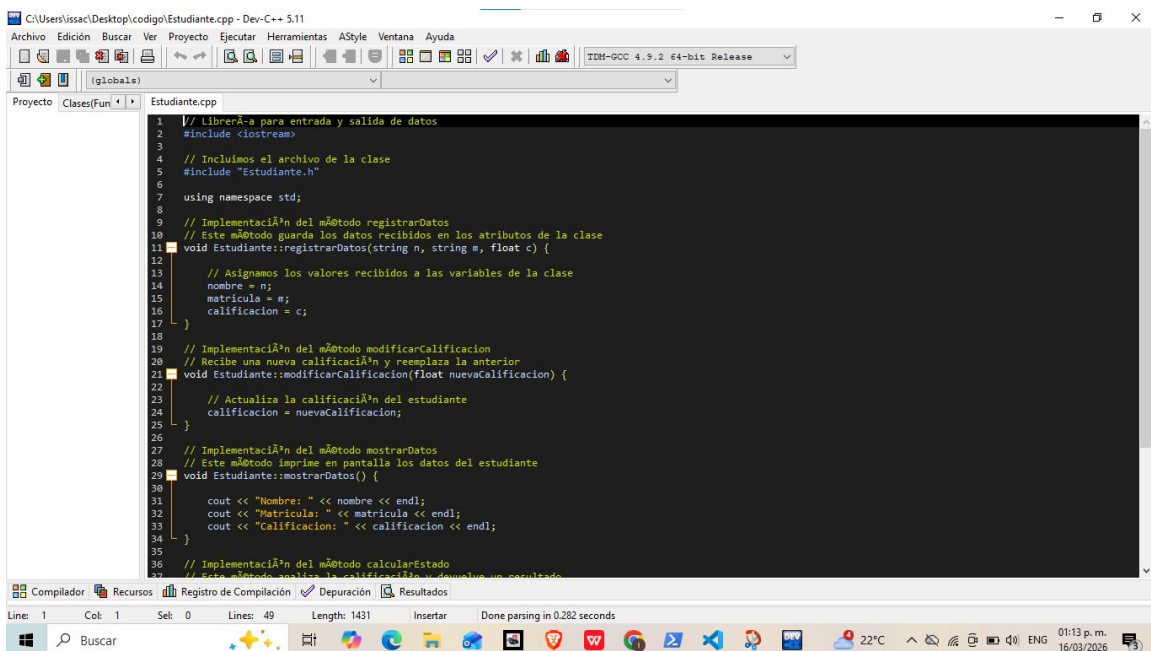
The image shows a Visual Studio Code editor window with the file 'Estudiante.h' open. The code defines a C++ class 'Estudiante' with private attributes 'nombre', 'matricula', and 'calificacion', and public methods 'registrarDatos', 'modificarCalificacion', and 'mostrarDatos'. The editor interface includes a sidebar with file explorer, search, and source control views, and a status bar at the bottom showing 'Ln 1, Col 1' and 'Spaces: 4'.

```
1 // Evita que el archivo se incluya varias veces en el programa
2 #ifndef ESTUDIANTE_H
3 #define ESTUDIANTE_H
4
5 // Librería para usar variables tipo texto (string)
6 #include <string>
7 using namespace std;
8
9 // Declaración de la clase Estudiante
10 class Estudiante {
11
12 private:
13     // Atributos de la clase (información del estudiante)
14     string nombre; // Guarda el nombre del estudiante
15     string matricula; // Guarda el número de control
16     float calificacion; // Guarda la calificación del estudiante
17
18 public:
19     // Métodos que podrá usar el programa
20
21     // Método para registrar los datos del estudiante
22     // Recibe parámetros: nombre, matrícula y calificación
23     void registrarDatos(string n, string m, float c);
24
25     // Método para modificar la calificación
26     // Recibe un parámetro con la nueva calificación
27     void modificarCalificacion(float nuevaCalificacion);
28
29     // Método para mostrar la información del estudiante
30     void mostrarDatos();
31 }
```

Explicación: En esta imagen se muestra la declaración de la clase Estudiante, donde se definen los atributos nombre, matrícula y calificación. También se declaran los métodos que permitirán registrar datos, modificar la calificación, mostrar información y calcular el estado del estudiante.

2. Archivo de implementación (Estudiante.cpp)

En este archivo se implementa el funcionamiento de los métodos declarados en el archivo de encabezado. Aquí se encuentra la lógica del programa para modificar los datos, mostrar la información y calcular si el estudiante aprueba o reprueba.



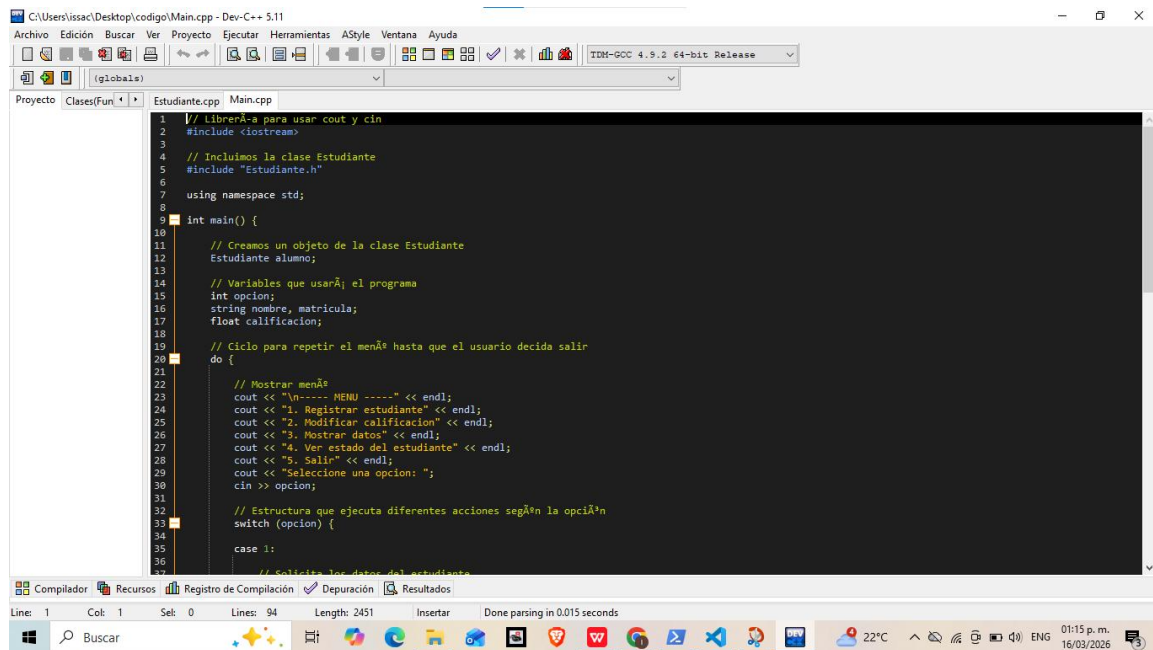
The image shows a Visual Studio Code editor window with the file 'Estudiante.cpp' open. The code implements the methods declared in 'Estudiante.h': 'registrarDatos' assigns input values to class attributes, 'modificarCalificacion' updates the 'calificacion' attribute, and 'mostrarDatos' prints the student's information. The editor interface includes a sidebar with file explorer, search, and source control views, and a status bar at the bottom showing 'Line: 1 Col: 1 Sel: 0 Lines: 49 Length: 1431'.

```
1 // Librería para entrada y salida de datos
2 #include <iostream>
3
4 // Incluimos el archivo de la clase
5 #include "Estudiante.h"
6
7 using namespace std;
8
9 // Implementación del método registrarDatos
10 // Este método guarda los datos recibidos en los atributos de la clase
11 void Estudiante::registrarDatos(string n, string m, float c) {
12
13     // Asignamos los valores recibidos a las variables de la clase
14     nombre = n;
15     matricula = m;
16     calificacion = c;
17 }
18
19 // Implementación del método modificarCalificacion
20 // Recibe una nueva calificación y reemplaza la anterior
21 void Estudiante::modificarCalificacion(float nuevaCalificacion) {
22
23     // Actualiza la calificación del estudiante
24     calificacion = nuevaCalificacion;
25 }
26
27 // Implementación del método mostrarDatos
28 // Este método imprime en pantalla los datos del estudiante
29 void Estudiante::mostrarDatos() {
30
31     cout << "Nombre: " << nombre << endl;
32     cout << "Matricula: " << matricula << endl;
33     cout << "Calificacion: " << calificacion << endl;
34 }
35
36 // Implementación del método calcularEstado
37 // Este método analiza la calificación y devuelve un resultado
```

Explicación: En esta imagen se muestra la implementación de los métodos de la clase. Cada método define cómo funciona cada acción del programa, como registrar datos, modificar la calificación o determinar si el estudiante aprobó o reprobó.

3. Programa principal (main.cpp)

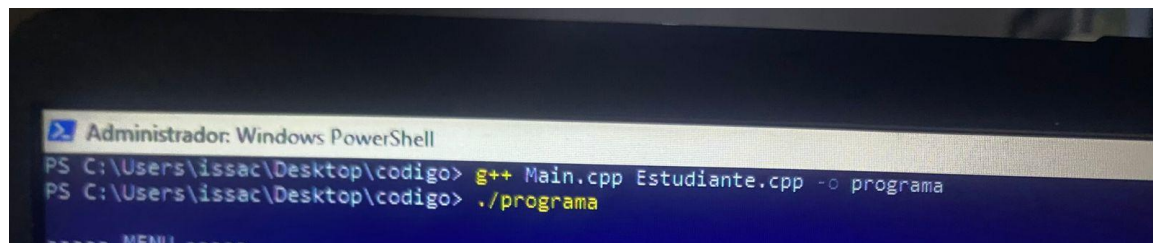
El programa principal es el encargado de ejecutar el sistema. En este archivo se crea un objeto de la clase Estudiante, se muestra un menú de opciones al usuario y se llaman los métodos correspondientes según la opción seleccionada.



```
1 // Librería para usar cout y cin
2 #include <iostream>
3
4 // Incluimos la clase Estudiante
5 #include "Estudiante.h"
6
7 using namespace std;
8
9 int main() {
10     // Creamos un objeto de la clase Estudiante
11     Estudiante alumno;
12
13     // Variables que usará el programa
14     int opcion;
15     string nombre, matricula;
16     float calificacion;
17
18     // Ciclo para repetir el menú hasta que el usuario decida salir
19     do {
20
21         // Mostrar menú
22         cout << "\n----- MENU -----" << endl;
23         cout << "1. Registrar estudiante" << endl;
24         cout << "2. Modificar calificación" << endl;
25         cout << "3. Mostrar datos" << endl;
26         cout << "4. Ver estado del estudiante" << endl;
27         cout << "5. Salir" << endl;
28         cout << "Seleccione una opción: ";
29         cin >> opcion;
30
31         // Estructura que ejecuta diferentes acciones según la opción
32         switch (opcion) {
33             case 1:
34                 // Solicita los datos del estudiante
35
36                 // Solicita los datos del estudiante
37
38         }
39     } while (opcion != 5);
40 }
```

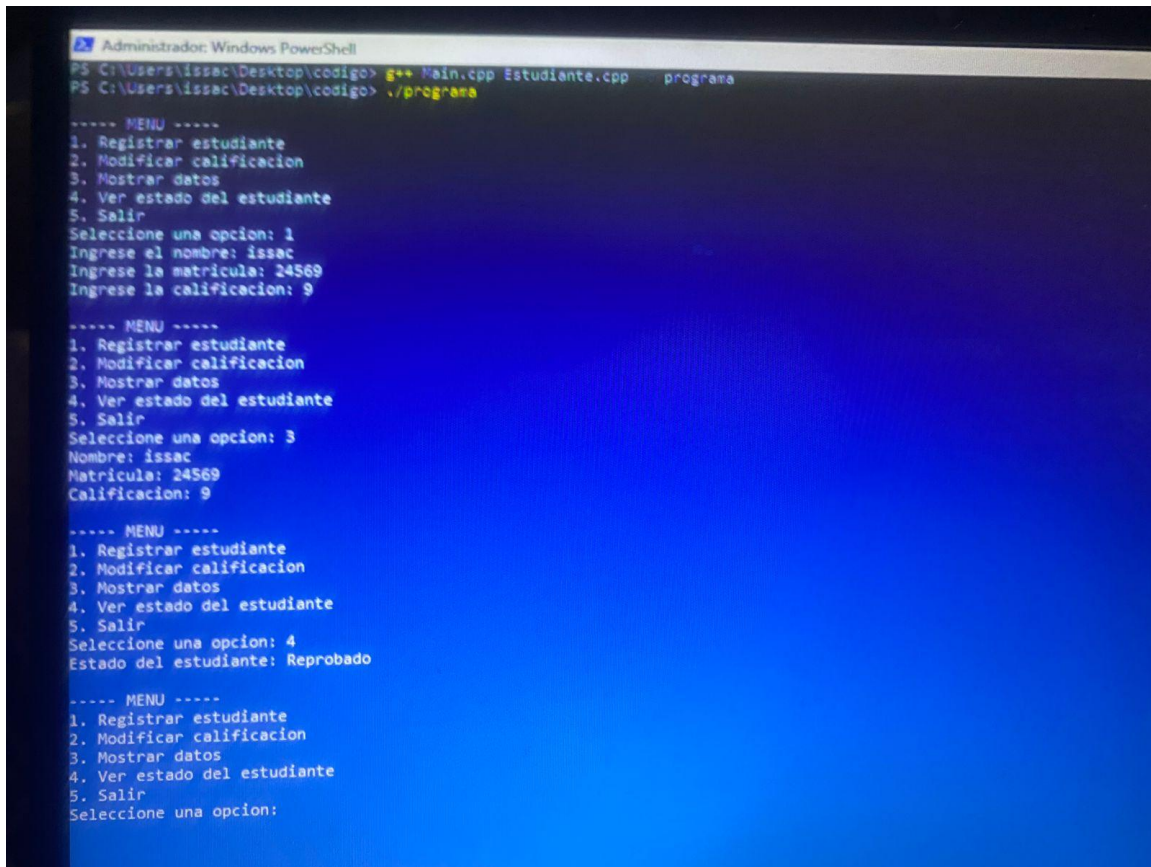
Explicación: En esta imagen se observa el programa principal donde se crea el objeto de la clase Estudiante y se implementa un menú interactivo que permite al usuario registrar datos, modificar la calificación, mostrar la información y consultar el estado del estudiante.

Compilación del programa



Explicación: En esta imagen se muestra el proceso de compilación del programa en el entorno de desarrollo. Esto indica que el código fue compilado correctamente sin errores.

Ejecución del programa



```
Administrador: Windows PowerShell
PS C:\Users\issac\Desktop\codigo> g++ Main.cpp Estudiante.cpp programa
PS C:\Users\issac\Desktop\codigo> ./programa

----- MENU -----
1. Registrar estudiante
2. Modificar calificación
3. Mostrar datos
4. Ver estado del estudiante
5. Salir
Seleccione una opción: 1
Ingrese el nombre: issac
Ingrese la matricula: 24569
Ingrese la calificación: 9

----- MENU -----
1. Registrar estudiante
2. Modificar calificación
3. Mostrar datos
4. Ver estado del estudiante
5. Salir
Seleccione una opción: 3
Nombre: issac
Matricula: 24569
Calificación: 9

----- MENU -----
1. Registrar estudiante
2. Modificar calificación
3. Mostrar datos
4. Ver estado del estudiante
5. Salir
Seleccione una opción: 4
Estado del estudiante: Reprobado

----- MENU -----
1. Registrar estudiante
2. Modificar calificación
3. Mostrar datos
4. Ver estado del estudiante
5. Salir
Seleccione una opción:
```

Explicación: En esta imagen se observa el menú principal del programa donde el usuario puede seleccionar diferentes opciones para interactuar con el sistema.

Conclusión

Esta práctica permitió comprender cómo se utilizan las clases y los métodos en programación orientada a objetos. También se aprendió cómo enviar información a los métodos mediante parámetros y cómo obtener resultados mediante valores de retorno. Además, se aplicó el concepto de programación modular separando el programa en distintos archivos.